ARIADNE: A General Purpose Indoor Navigation System

Athanasios Tsoukleidis-Karydakis (thanos.karidakis@gmail.com)
Dimitrios-David Gerokonstantis (dimitrisger2@gmail.com)
Spyridon Papadopoulos (spyrospapadopoulos21@gmail.com)
Savvina Nastou (savnastou@gmail.com)
Filippos Sevastakis (philipsevas@yahoo.gr)
Dionisios Rafail Deuteraios (defteraios.dionisis@gmail.com)
Epimenidis Kountourakis (minoskoun@gmail.com)
Konstantinos Ioannou (kostisioannou101616@gmail.com)
Maria Zerva (mariazerva001@gmail.com)

National Technical University of Athens - School of Electrical and Computer Engineering
Advisor: **Georgios Goumas** (goumas@cslab.ece.ntua.gr)

***Abstract*—Nowadays, navigation systems are prominent in everyday life, facilitating the way people go about their daily routines. Although outdoor navigation systems are abundant and widely used, indoor navigation remains an emerging technology, with existing systems often tailored to specific indoor environments. This project aims to develop an innovative and universal indoor navigation system, applicable for use in general environments. The proposed system employs advanced floor plan analysis algorithms to identify and extract key structural components (rooms, corridors, doors etc.) from building floor plans. User localization within a given building is achieved through the use of BLE beacons installed on-site, enabling functionality without the need for an Internet connection. Additionally, a mobile application is developed so that the user is able to navigate within each building through a friendly User Interface. By utilizing this application, users can navigate any indoor environment by simply supplying the corresponding floor plan.**

***Index Terms*—Indoor Navigation, BLE beacons, Floor Plan, React Native, Javascript, Python**

I. Introduction and Systems Overview

Global Positioning Systems (GPS) have brought a revolutionary change to the way people navigate and are widely adopted for outdoor navigation purposes. However, such systems fall short on facilitating indoor navigation, since they are oriented to outdoor environments. At the same time, existing indoor navigation systems are tailor-made for specific environments and usually proprietary. Driven by the need to overcome these limitations, in this project we introduce **ARIADNE**, an innovative and open-source indoor navigation system provided through a mobile application. The system's architecture consists of many interconnected components, which are presented and described below.

To begin with, an **admin User Interface** (web application) is provided, through which the registration of a new building is carried out by its administrator. Specifically, the administrator uploads the floor plan of the building (either on digital or printed format) and subsequently the **floor plan analysis algorithm** detects the key components of the building and generates a graph with nodes which discretize the available space and edges representing human-walkable paths. Afterwards, the results are presented to the administrator through the User Interface, followed by a refinement process that allows the administrator to make corrections or additions (for instance, in case of a misidentified object).

After the inference procedure has been completed, the resulting annotated floor plan is uploaded to a **Content Management System (Strapi)**, through which the administrators of the application can manage the available data. In addition, the Content Management System provides the buildings' data to the **mobile application** so that the users can download the buildings they want to navigate through. After downloading a new building, the user can select it and an interactive map is presented. Through this map, the users are able to :

- **view their position in the building.**
  The mobile app periodically receives signals (packets) from the BLE beacons and runs a localization algorithm to identify the user's position.
- **inspect the components of the building, i.e. rooms, corridors, doors, emergency (and other) exits.**
  A floor plan analysis is employed resulting to the identification of such components, which are presented to the user through the interactive map.
- **select a destination to navigate to and follow the identified path**
  The application runs a shortest path algorithm to determine the route the user must follow to reach the desired destination.

Figure 1 illustrates the component architecture of the indoor navigation system as it was described above.

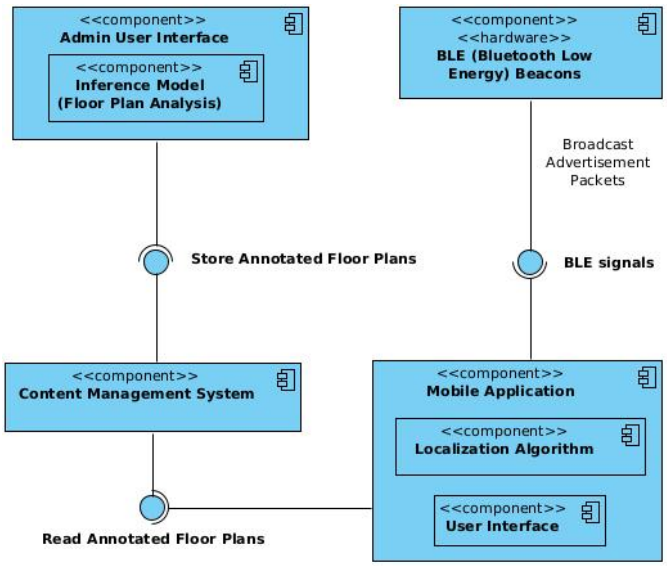


Fig. 1: Component Architecture Diagram

II. Beacons

A. Beacons for Indoor Positioning

Beacons have emerged as a key technology in numerous Internet of Things (IoT) applications, particularly in the realm of indoor navigation systems, as these compact devices are able to transmit periodic signals that can be detected by nearby mobile devices. Beacons enable location estimation through the Received Signal Strength Indicator (RSSI) [1], which reflects the strength of the signal from the beacon to the mobile device. By integrating RSSI data from multiple beacons, the position of a device can be triangulated, typically with an accuracy within a few meters. This combination of signal strength measurements and multi-beacon deployment is essential for precise indoor localization, making beacons a key component in developing effective indoor navigation systems. Bluetooth Low Energy (BLE) beacons are favored for their energy efficiency, consuming minimal power while providing essential proximity data. This makes them ideal for scenarios where continuous transmission is needed, as they can operate on a single battery over extended periods of time depending on the configuration of their settings. The key challenge in deploying BLE-based beacon systems is balancing between power consumption and positioning accuracy. The transmission power of each beacon, along with parameters like the advertising interval and the scanning interval, must be carefully optimized to minimize energy usage while ensuring sufficient signal coverage. Furthermore, in large-scale deployments, it is important to minimize interference between beacons and avoid over-provisioning, which could increase both installation and operational costs. For this study, the Nordic Semiconductor nRF52840 SoC is used due to its energy efficiency, strong BLE support, and flexible transmission power. These features allow fine-tuned control over range and energy consumption, making it ideal for large-scale BLE deployments. While several other BLE chips, such as Nordic's nRF54L and Renesas' DA14531, also offer similar capabilities, the nRF52840 was selected for its strong performance in BLE mode and its suitability for large-scale deployments. In the subsequent sections, we will explore operational modes of the beacon system, including dynamic adjustments to key parameters that ensure low-power but still reliable indoor positioning.

B. Features

The beacons we decided to create are based on the nRF52840 SoC from Nordic. This SoC is built around the 32-bit ARM Cortex-M4 CPU and has support for Bluetooth 5.4, Matter, Thread, Zigbee and Bluetooth Mesh. We chose this particular SoC for its great performance/watt and for its mesh capabilities. At the current state of the project, we have not taken advantage of any of these capabilities in our project but we are planning to do so in order to provide detailed heatmaps of foot traffic patterns, track unusual patterns (e.g., bottlenecks or unauthorized access) and much more.

C. Parameters to consider

The number of beacons per area and the determination of their coordinates are critical parameters in achieving cost and energy efficiency while maintaining positioning accuracy. A very small number of beacons per area, though preferable in terms of installation cost, could have a significant impact on accuracy. The transmission power of each beacon, measured in dB, is also closely related to these parameters. Higher transmission power will result in longer transmission range. However, it will also lead to higher power consumption per beacon and to higher interference between neighboring beacons, as the signal can reach further coordinates, thus affecting accuracy [2]. A positioning arrangement that allows for the smallest number of beacons per area with the lowest transmission power possible is crucial for optimizing energy performance. The advertising interval is the amount of time between two consecutive advertising packet transmissions. It ranges between 20 ms and 10.24 s in steps of 0.625 ms. The longer the advertising interval of each beacon, the lower its power consumption. Maintenance of the desired accuracy could be reached by moderately increasing the scanning window and/or decreasing the scanning interval of the user's device, while not severely draining its battery.

D. Our approach

In most implementations, BLE chips function as **broadcasters**, emitting data at predefined intervals (**advertising intervals**), regardless of whether the information is needed. When not transmitting, they enter **sleep mode** to save energy. However, beacons may not be used for a significant period of time when no one crosses a particular area of the building. During this time, beacons are redundant. In this case continuous broadcasting with fixed advertising interval becomes unnecessary and wastes energy. The proposed solution is to divide the operations into three dynamic **modes (states)** that adjust dynamically based on environmental conditions [3], [4].

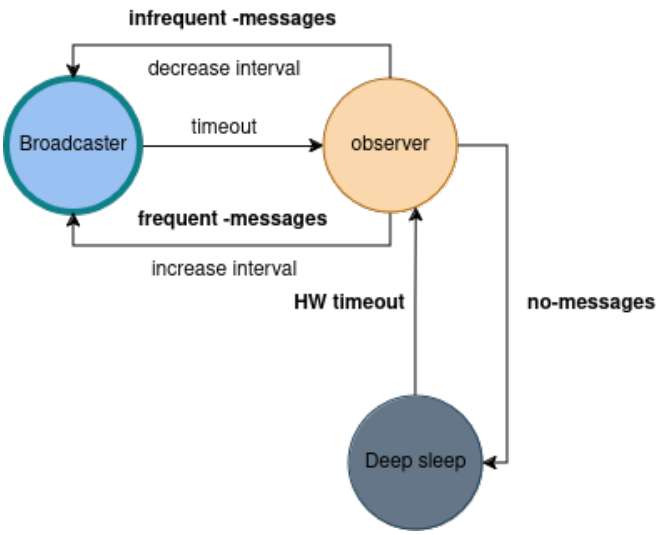


Fig. 2: Beacon State Diagram

E. States and Modes of Operation

1) Broadcaster Mode (Classic Beacon): In this mode, the BLE chip periodically broadcasts advertising data at a dynamically adapted interval. The interval starts with a predefined value and can dynamically adjust within a defined range. The primary goal of this mode is to transmit advertising packets, enabling a mobile device to estimate its distance from the beacon using the Received Signal Strength Indicator (RSSI).

2) Observer Mode: In Observer Mode, the chip performs a scanning operation, where the parameters of the scanning interval and scanning window can change dynamically. The purpose of this mode is to receive incoming advertising packets from nearby user devices (e.g. smartphone). By analyzing these packets, the chip gathers information about the surrounding environment, such as traffic, and adjusts the parameters of other states accordingly.

3) Deep Sleep Mode: When the chip is in Deep Sleep Mode, it enters a low-power state where most of its operations are paused. The chip periodically wakes up using a hardware timer, and the wake-up period is configurable as a parameter. The main objective of this mode is to conserve energy by suspending unnecessary operations when no transmission or scanning is needed. The chip only wakes-up periodically to switch to observer mode and check specific events or conditions, reducing overall power consumption.

F. Adjustable Parameters

For dynamic energy management, the following parameters need adjustment:

1. **Advertising Interval** (Broadcaster Mode): Affects responsiveness and energy consumption.
2. **Scanning Interval** and **Scanning Window** (Observer Mode): Impact packet reception and energy usage.
3. **HW Timer Period** (Deep Sleep Mode): Controls sleep and wake-up duration.

G. State Transition and Parameter Optimization

The goal is to ensure that each state operates with the minimum set of parameters required, as these directly impact both energy efficiency and responsiveness. Additionally, transitions between states must be managed carefully, as frequent switching, especially between simple and deep active modes (e.g., Broadcaster or Observer), can become highly energy-costly.

III. Localization

A. Objective

To be useful in real world indoor environments, the estimated location of the user's device must be within 2-3 meters of its actual location. Our goal is to achieve an horizontal accuracy of at most 1 meter, as larger errors may cause confusion to the user in cases when there are multiple doors or corridors with small distances between them. The vertical coordinate should be estimated accurately enough to specify which floor the user is on, so we aim for an accuracy of around 1.5 meter. Moreover, the navigation app will constantly display the user's location, thus the consecutive estimations should form a smooth path without notable position jumps or abrupt direction changes as these kind of movements are unlikely to occur during walking.

B. Techniques used

As Bluetooth signals display varying strength and range due to interference with various obstacles as well as humans and other signals, it is necessary to combine many computational approaches in order to eliminate some of the possible variance in location estimations. Such approaches include:

- **RSSI:** Since the system's beacons produce signals of constant strength and these signals get weaker the further they are from the transmitter, the distance between the user's device and any specified beacon can be approximately obtained by measuring the signal's power loss when reaching the device. By gathering multiple signals from different beacons we apply trilateration techniques to estimate the user's location.
- **Fingerprinting:** In an indoor environment with stable WiFi and Bluetooth transmitters the user's location can be estimated just by knowing which of these signals are being received by the device. To achieve this, during the setup of the positioning system a mobile device maps the signals that are reachable from any location of the building, and from this mapping a database is constructed that contains almost any signal combination and the location that it corresponds to.
- **History-Based Prediction:** It can be assumed that at any moment the user's moving speed and direction remain relatively constant. By constantly measuring these parameters using previous estimations as well as the device's accelerometer, we create the path that is taken during

the past seconds of the user's movement. The possible position of the user is predicted to smoothly extend this path.

To properly combine all the methods used, the harmonic mean of the estimations and other averaging techniques are used to remove outliers and to reduce the variance of each of the above techniques.

IV. Navigation

A. Aim and Objectives

To achieve indoor navigation, it is essential to construct a graph of the building. This graph will be utilized to find the shortest path between two points using the A* algorithm. To construct the graph, the user must provide the floor plan of the building. Currently, we are focusing on supporting only floor plans in the form of images. We also intend to automatically handle floor plans provided as AutoCAD files.

The main objectives of this part of the project are as follows:

- Wall and obstacle detection
- Key components detection and labeling (doors, rooms, stairs, elevators, etc.)
- Graph construction
- Preprocessing and route caching

B. Image Processing

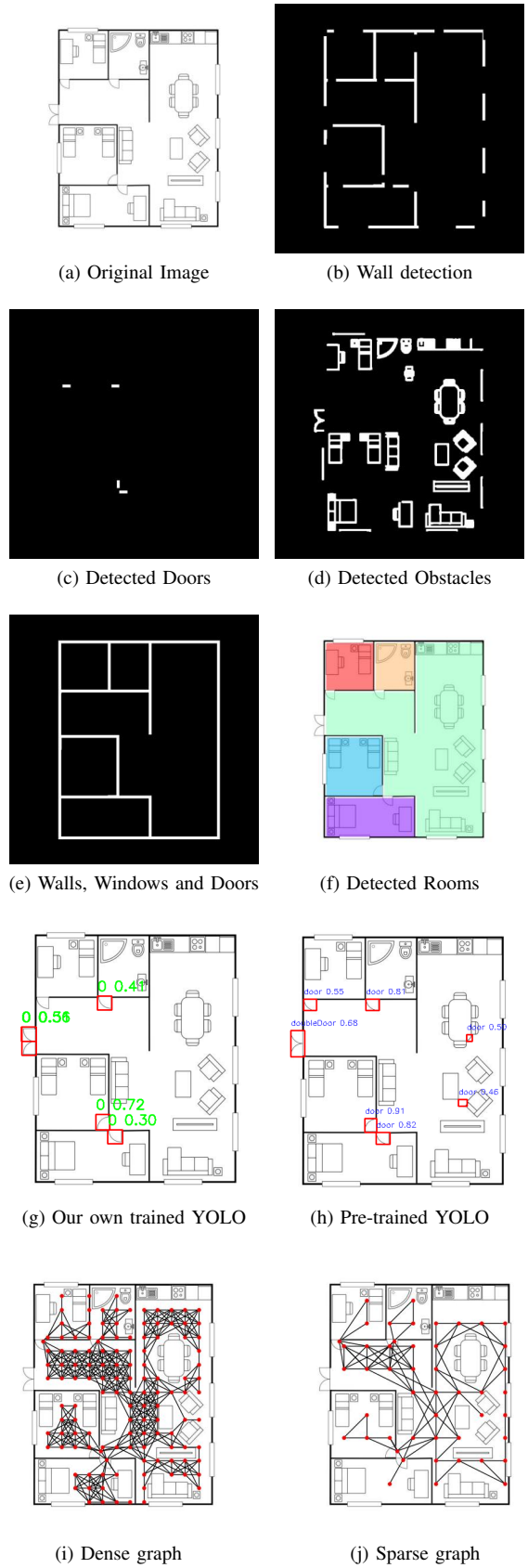
The primary challenge in graph construction is processing the image of the floor plan. At start, we attempted to process a simple floor plan image, as shown in Figure 3a.

1) Wall and Obstacle Detection: First, we need to detect the building area. To identify the free space inside the building, we must detect the walls and obstacles. At this stage, to do so, we employ classic computer vision techniques, such as morphological operations, edge detection, contour detection, and thresholding. We lie on the assumption that the walls appear as more solid black lines in the image, while the obstacles appear as more gray and less solid shapes.

In Figure 3b, we observe that we manage to detect the walls in the floor plan image; however some parts are missing.

2) Door Detection: Door detection is crucial for constructing the graph. We need to detect the doors to identify the connections between rooms. We focus on using Deep Learning techniques for the door identification, especially Convolutional Neural Networks (CNNs). Specifically, for object detection, we utilize the bounding boxes approach of the YOLO CNN algorithm. In figures 3g, 3h, the results of two attempts at door detection are presented. The first one makes use of a YOLO model trained by us while the second one leverages an existing, pre-trained model. The quality of these results is satisfying and further improvements will be made in the future.

In Figure 3c, we show the detected doors in the floor plan image, applying only simple image processing techniques. We see that we manage to detect only the single doors and not the double doors. Moreover, this approach is not robust and extremely unreliable. We assumed that the doors appear at the new connections that are created by dilating the walls.



(a) Original Image (b) Wall detection

(c) Detected Doors (d) Detected Obstacles

(e) Walls, Windows and Doors (f) Detected Rooms

(g) Our own trained YOLO (h) Pre-trained YOLO

(i) Dense graph (j) Sparse graph

Fig. 3: Simple Floor Plan Image Processing

3) Window Detection: Windows are also important for constructing the graph as they should be treated as walls or obstacles. While windows do not appear in floor plans with thick black lines, we try to handle them as obstacles.

4) Room Detection: After detecting walls, windows and doors, we can detect the rooms by finding the contours of the free space, employing computer vision techniques. An alternative approach could utilize instance segmentation CNN techniques.

In Figure 3f, we show the rooms are correctly detected in the floor plan image.

C. Graph Construction

Having extracted the key components of the floor plan, the human-walkable space of the building is then known. By discretizing this space, we construct the aforementioned graph, which is configurable and can either be dense (fig. 3i) or sparse (fig. 3j).

D. Admin User Interface

After the inference procedure has been completed, the annotated floor plan is then presented to the user through the admin UI (fig. 4).

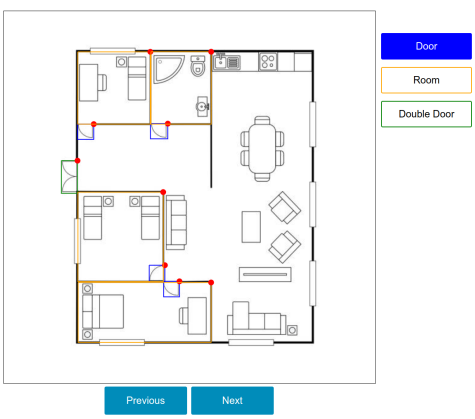


Fig. 4: Admin User Interface

This UI supports a refinement process, through which the user can delete misidentified objects, add omitted objects and label the rooms (e.g. CS Lab, Restaurant). We also plan to develop a preliminary room labeling process based on the floor plan legend by utilizing the OCR technology.

E. Challenges

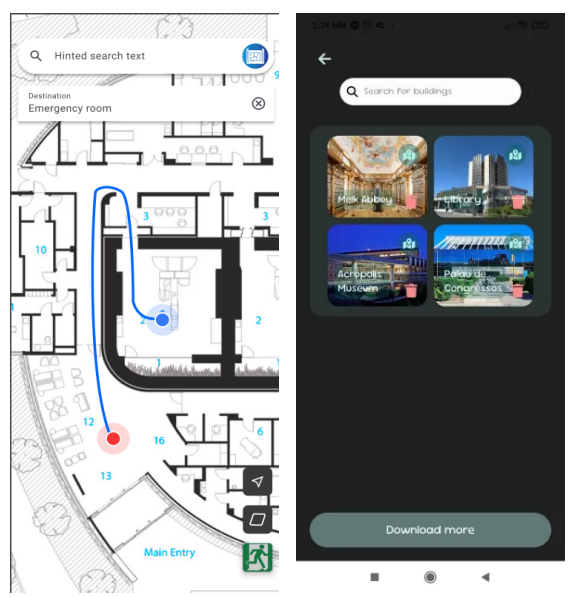
Some of the challenges that were met during the development process are presented below:

- The need to recognize all possible door types, as well as stairs and elevators complicate the training phase. For instance, the sliding doors closely resemble walls.
- In case of rotated doors, the current annotation technique could be enhanced by employing oriented bounding boxes.
- Computer vision approach can become more complex when dealing with curved walls and windows.

V. Mobile Application

The mobile application consists of three pages:

1) **Main Page: Interactive Map** [Figure 5a]
This page displays the interactive map of the selected building and shows the user's position in real time. The user can inspect all possible destinations (as identified by the analysis), select the desired one and view the route to this destination. The user can also select the "Emergency Exit" option to find the nearest exit and the route to reach it.
2) **Library** [Figure 5b]
This page displays the locally saved buildings. The user can search among these buildings, delete a building or select one to navigate through. Furthermore, for each building, the user can be redirected to its location in Google Maps by pressing on the corresponding button.
3) **Download More Page**
By pressing on the "Download More" button of the "Library" page, the user is redirected to this page, which displays all the available buildings (stored in the CMS). The users can download the desired building and store it to their Library. This process is the only one in the mobile application requiring an internet connection.



(a) Main Page (from Figma) (b) Library

Fig. 5: Mobile Application - Pages

The mobile application prototype has been developed using **Figma** and is accessible here. The mobile application is developed using **React Native** and the **Expo** platform.

VI. Use Cases and Future Directions

ARIADNE is a general purpose application and as a result can be utilized in any indoor navigation use case. These include but are not limited to : Healthcare facilities (e.g. Hospitals, Clinics), Educational Institutions (e.g. Universities),

Transport facilities (e.g. airports, subway stations), Museums, Libraries, Event venues, Shopping malls and many more.

Furthermore, the methods developed for this project can be applied to deal with more specific and advanced navigation problems. For instance, this technology can be extended to assist disabled or elderly individuals in navigating through indoor spaces or facilitate localization and path planning tasks for autonomous vehicles or robotic systems.

Additionally, the application could also provide some customization features. For instance, the administrator of a building could be able to enrich the automatic annotation information with additional details for each particular room (e.g. details about the exhibits of a particular museum room).

ACKNOWLEDGMENT

We would like thank **Nordic Semiconductor ASA** for providing us with the necessary hardware for this project.

REFERENCES

[1] P. Spachos, I. Papapanagiotou and K. N. Plataniotis, "Microlocation for Smart Buildings in the Era of the Internet of Things: A Survey of Technologies, Techniques, and Approaches," in IEEE Signal Processing Magazine, vol. 35, no. 5, pp. 140-152, Sept. 2018, doi: 10.1109/MSP.2018.2846804.

[2] P. Spachos and K. Plataniotis, "BLE Beacons in the Smart City: Applications, Challenges, and Research Opportunities" in IEEE Internet of Things Magazine, vol. 3, no. 1, pp. 14-18, March 2020, doi: 10.1109/IOTM.0001.1900073.

[3] D. Ye and M. Zhang, "A Self-Adaptive Sleep/Wake-Up Scheduling Approach for Wireless Sensor Networks," in IEEE Transactions on Cybernetics, vol. 48, no. 3, pp. 979-992, March 2018, doi: 10.1109/TCYB.2017.2669996.

[4] X. Cao, J. Wang, Y. Cheng and J. Jin, "Optimal Sleep Scheduling for Energy-Efficient AoI Optimization in Industrial Internet of Things," in IEEE Internet of Things Journal, vol. 10, no. 11, pp. 9662-9674, 1 June1, 2023, doi: 10.1109/JIOT.2023.3234582.